

N-Body Simulation Performance Analysis: From Python Algorithms to RISC-V Vectorization

Hafsa Tanveer

SMCS, IBA

29041

h.tanveer.29041@khi.iba.edu.pk

Manal Fatima

SMCS, IBA

29018

m.fatima.29018@khi.iba.edu.pk

Areeba Salman

SMCS, IBA

29075

a.salman.29075@khi.iba.edu.pk

Abstract—This paper explores the evolution of an N-Body simulation, starting from a foundational naive algorithm using Leapfrog integration, and progressing to a highly optimized Barnes-Hut implementation. We discuss our journey bridging high-level Python logic with low-level RISC-V assembly, highlighting the trade-offs between computational complexity and hardware-level vectorization.

Index Terms—N-body, Leapfrog, Euler, Barnes-Hut, RISC-V, Vectorization

I. INTRODUCTION

The N-body problem calculates how a specific number of bodies interact within a given system, a computational model heavily relied upon in fields ranging from astrophysics to plasma physics. We initially tackled this problem using a straightforward, naive algorithm. Ultimately, we shifted our focus to improving time efficiency by transitioning from the naive $O(n^2)$ algorithm to the $O(n \log n)$ Barnes-Hut algorithm, exploring spatial partitioning via octrees.

II. NAIVE IMPLEMENTATION AND SOFTENING

The naive N-body algorithm accurately calculates gravitational forces but suffers from an $O(n^2)$ time complexity because it computes the interaction of every single object with every other object.

A critical component we integrated into our force calculation was the gravitational softening parameter. In a discrete time-step environment, two particles can sometimes drift too close to one another, driving the distance mathematically close to zero and causing the calculated force to explode toward infinity. By introducing a softening term (ϵ), we smoothed out the gravitational potential at tight ranges, successfully eliminating division-by-zero errors without sacrificing accuracy over larger distances.

III. BARNES-HUT OPTIMIZATION

To scale our simulation, we implemented the Barnes-Hut algorithm, which drops the computational complexity down to $O(n \log n)$ by grouping distant clusters of bodies and approximating them as a single virtual mass at their center of mass.

```
hafsalinus@HafsaASUS:~/nbody/milestone_three$ make run
qemu-riscv64 -cpu rv64,v=true,vlen=128 ./nbody
vector version is not specified, use the default value v1.0
Loaded 100 particles from stable_random_system100.csv
Step 0 E = -3.96118306e+34 dE/E0 = +0.0000e+00
Step 500 E = -3.96114663e+34 dE/E0 = +9.1958e-06
Step 1000 E = -3.96115730e+34 dE/E0 = +6.5023e-06
Step 1500 E = -3.96118165e+34 dE/E0 = +3.5470e-07
Step 2000 E = -3.96112655e+34 dE/E0 = +1.4266e-05
Step 2500 E = -3.96112007e+34 dE/E0 = +1.5902e-05
Step 3000 E = -3.96115431e+34 dE/E0 = +7.2580e-06
Step 3500 E = -3.96113481e+34 dE/E0 = +1.2179e-05
Step 4000 E = -3.96109590e+34 dE/E0 = +2.2002e-05
Step 4500 E = -3.96111660e+34 dE/E0 = +1.6777e-05
Step 5000 E = -3.96114457e+34 dE/E0 = +9.7156e-06
Step 5500 E = -3.96110995e+34 dE/E0 = +1.8455e-05
Step 6000 E = -3.96110872e+34 dE/E0 = +1.8766e-05
Step 6500 E = -3.96115326e+34 dE/E0 = +7.5229e-06
Step 7000 E = -3.96113982e+34 dE/E0 = +1.0916e-05
```

Fig. 1. Result of Naive Code

A. Octree Spatial Partitioning

We achieved this spatial optimization using an octree. To make the eventual translation to RISC-V assembly smoother, we designed our Python prototype (Barnes_hut_refactored.py) using global flat arrays for the node pool (e.g., node_cx, node_mass, node_child) rather than object-oriented classes. The root node represents our entire universe, and internal nodes recursively subdivide the space into eight child octants until every leaf contains at most a single particle.

During traversal, if the ratio of a cell's size (s) to its distance from the body (d) is less than a designated threshold (θ), the traversal stops, and the entire cluster is approximated as one body.

IV. HARDWARE ACCELERATION: RISC-V IMPLEMENTATIONS

Bridging our algorithm to bare-metal RISC-V assembly provided unique insights into hardware optimization.

For the Barnes-Hut algorithm (Barnes_hut_scalar.s), we mapped our Python flat arrays directly into memory data sections. We meticulously translated the recursive logic of tree construction and traversal into scalar floating-point instructions, dynamically managing a bounded pool of 900 nodes in memory.

Conversely, for the naive algorithm (nbody.S), we utilized the RISC-V Vector (RVV) extensions. By stripping out branching logic and utilizing instructions like vle32.v

(vector load) and `vfmacc.vf` (fused multiply-accumulate), we processed gravitational interactions in parallel strips, vastly outperforming scalar execution for smaller particle counts.

V. CHALLENGES FACED

Developing both a physics engine and its corresponding assembly implementation presented multiple hurdles:

A. Barnes-Hut: Recursive Assembly Constraints

During the RISC-V assembly implementation of the Barnes-Hut algorithm, managing the call stack proved exceptionally difficult. Specifically, within our `insert_particle` function, we initially failed to save the return address register (`ra`) to the stack before triggering a recursive call to the `subdivide` function. Because the return path was overwritten, the execution flow was lost entirely, resulting in infinite loops and segmentation faults. We resolved this by carefully designing stack frames to preserve `ra` and caller-saved registers across all recursive depth levels. Furthermore, we had to carefully calculate mass outside the subdivision loop to ensure the center of mass was accurate.

B. Naive Implementation: Energy Conservation

For the naive implementation, our primary issue was strict energy conservation. To fully resolve the energy conservation issues within our naive implementation, we had to refine the simulation’s time step duration (dt). Initially, a larger time step caused our discrete approximations to break down—especially during close planetary encounters where gravitational forces change rapidly—leading to numerical instability and artificial energy drift. By significantly reducing the dt duration, we increased the temporal resolution of the simulation. This simple but critical adjustment allowed the Leapfrog integrator to accurately capture sudden shifts in acceleration, ensuring that the system’s total energy remained strictly conserved over extended periods.

VI. CONCLUSION

This project served as a rigorous exercise in full-stack optimization, stretching from mathematical physics down to memory registers. We demonstrated that while RISC-V Vector extensions offer incredible speedups for parallelizable problems like direct summation, scaling simulations to massive proportions ultimately requires algorithmic brilliance, as seen in the Barnes-Hut tree logic.

ACKNOWLEDGMENTS

We thank Sir Salman Zaffar for his invaluable guidance and expertise, and our teaching assistant Hamza for his thankless help and resources.